

Demystifying Git

Alexandros Zacharakis

September 28, 2026

Intro

The Problem

report-final.tex

report-final-really.tex

report-final-v2.tex

report-final-really-fixed.tex

Questions

- ▶ What changed since yesterday?
- ▶ Which version did we send?
- ▶ Can I try something without breaking the current version?
- ▶ How do we combine my work with yours?
- ▶ What went wrong? *How?*

Git's Core Idea

Saving and showing the history of a project.



History

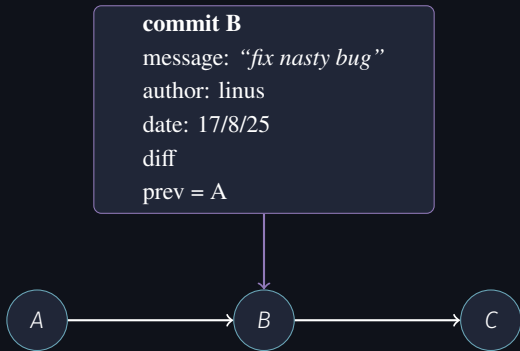
Each dot is a *saved snapshot* of the whole project at one moment.

Most Git commands answer one of three questions:

- ▶ Where am I?
- ▶ What changed?
- ▶ How do I combine with changes from others?

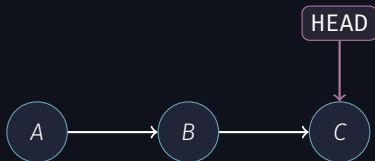
Commits

A commit is one saved point in the history.



Where Am I?

At any moment, Git has a current saved version.



HEAD

The version I am currently *looking* and *modifying*.

Working Files and Saving Work

Saved Version vs. Working Files

Git separates the version it has saved from the files you are editing.

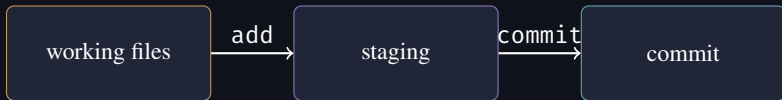


Question

What changed between the saved version and my working files?

Staging

Before saving a new version, you choose what should go into it.



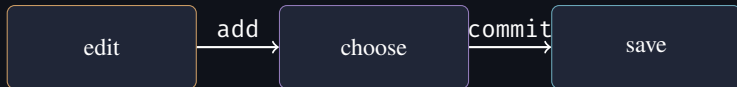
Mental Model

Staging is the draft of the next snapshot.

This lets you edit many things, but save only the coherent part.

The Basic Loop

The everyday Git loop is simple:



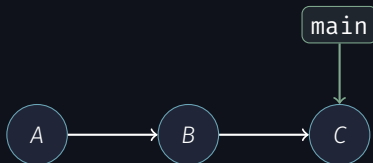
Commands as Questions

- ▶ `git status`: what is the state of my work?
- ▶ `git diff`: what changed?
- ▶ `git add`: what goes into the next snapshot?
- ▶ `git commit`: save the snapshot.

Branches

Branches

A branch is a name for a place in history.

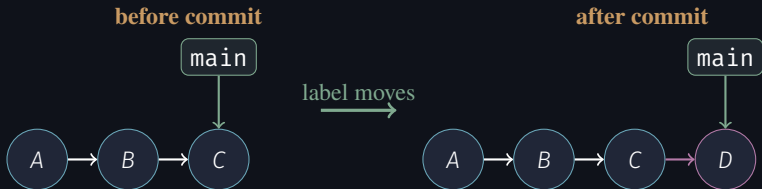


Branch

A movable label pointing to a saved version.

Committing Moves the Branch

Suppose we are on `main`.

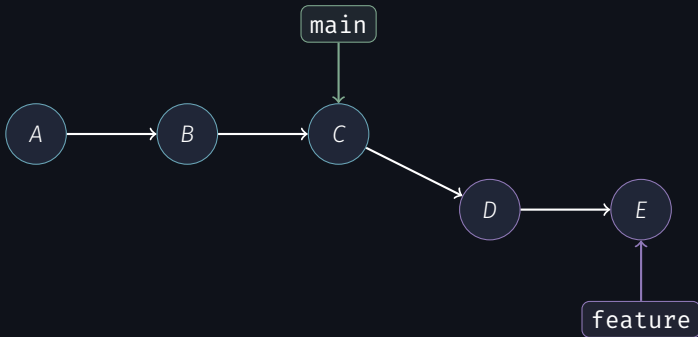


Committing

When you commit, Git creates a new saved version and moves the current branch name to it.

Trying Something Safely

Branches are useful because they let you try an idea without moving `main`.



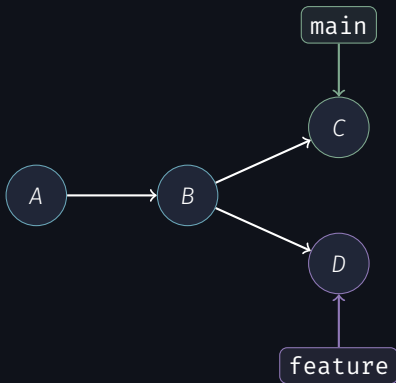
Feature branches

`main` stays at the stable version. `feature` moves while you experiment.

Merging and Conflicts

Merging

A merge combines work from another branch into the branch you are on.

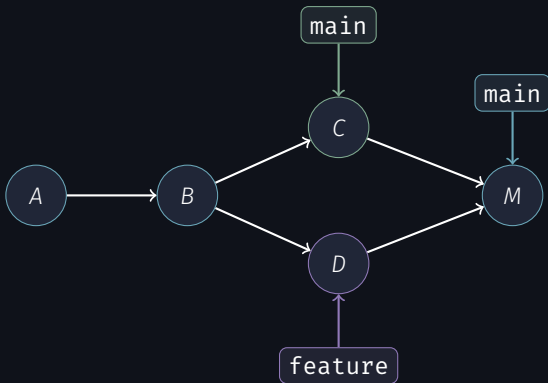


Merging

Create a saved version that contains the work from both timelines.

Merging

A merge combines work from another branch into the branch you are on.

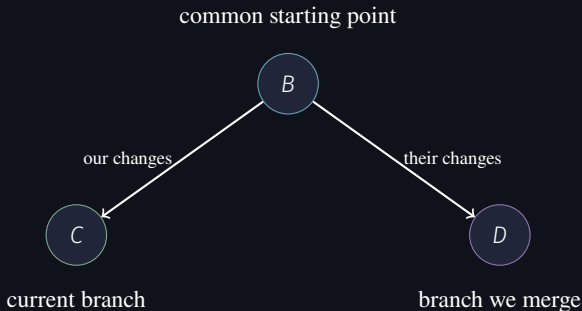


Merging

Create a saved version that contains the work from both timelines.

What Git Tries to Do

Git compares three versions:



How merging works

Git asks: from the common starting point, what did each side change? Then it tries to apply both sets of changes.

Easy Merge

If the changes touch different places, Git can combine them automatically.

Base

- ▶ intro
- ▶ theorem
- ▶ proof

Our branch

- ▶ intro
- ▶ better theorem
- ▶ proof

Their branch

- ▶ intro
- ▶ theorem
- ▶ clearer proof

Merge Result

- ▶ intro
- ▶ better theorem
- ▶ clearer proof

Conflict

A conflict happens when both timelines changed the same place differently.

Base

title: Git intro

Our branch

title: Git basics

Their branch

title: Git tutorial

Git's Question

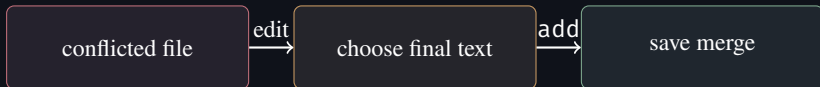
Which title should the merged version use?

Important

It's a *feature* not a bug! A conflict is not Git being broken. It is Git *refusing to guess*.

Resolving a Conflict

Resolving a conflict means editing the file into the version you actually want.



Handling Conflicts

After you resolve the file, `git add` means: *this conflict is resolved; use this version in the merge.*

Merge Checklist

When a merge feels confusing, ask:

- ▶ Which branch am I on?
- ▶ Which branch am I bringing in?
- ▶ What was the common starting point?
- ▶ Did both sides change the same place?
- ▶ What should the final version say?

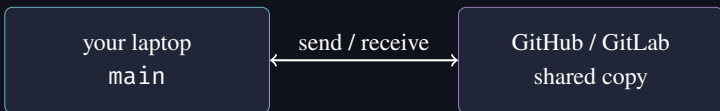
Takeaway

Merging is not magic. It is combining changes made since a shared starting point.

Remotes

Your Copy and the Shared Copy

Until now, everything happened on your laptop.



Remotes

Gitlab/Github etc are to git what Google Docs are to .doc files.

Clone

The first time you get a project, you copy the shared repository + history.

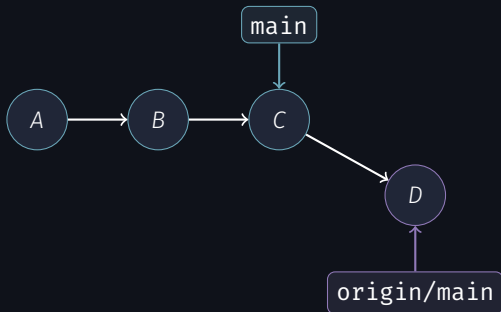


Cloning

make my own local copy, *including the saved history*.

Fetch

Other people may add commits to the shared copy.

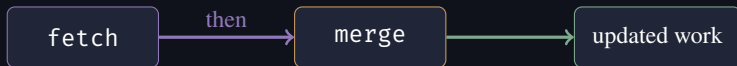


`git fetch`

Fetch asks the shared copy: *what new saved versions do you have?* It downloads them, but does not change your current work.

Pull

Usually, after receiving new commits, you want to integrate them.



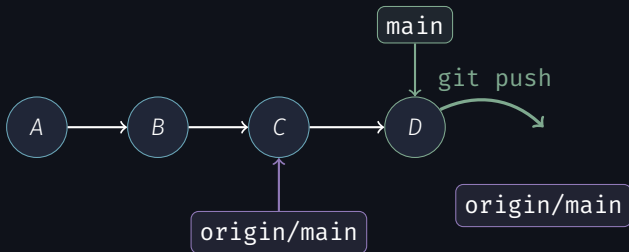
Pulling

```
git pull ≈ fetch + integrate into my branch
```

If both sides changed the same place, pulling can also create a conflict.

Push

When you have commits others do not have, you send them to the shared copy.



Pushing

push publishes your saved versions by moving the shared branch forward.

Why Push Can Be Rejected

Push is easy when the shared branch is behind your branch.

But suppose someone else already pushed:

`origin/main` has commits you do not have.

Rejected Push

Git refuses to overwrite the shared timeline with your older view of it.

The usual fix:

`git pull` $\leadsto$ `resolve/integrate` $\leadsto$ `git push`

Remote Checklist

When working with other people, ask:

- | | |
|---|--------------------|
| ▶ Do I need to receive their commits? | <code>fetch</code> |
| ▶ Do I want to integrate their commits now? | <code>pull</code> |
| ▶ Do I have commits to publish? | <code>push</code> |
| ▶ Did my push fail because the shared copy moved? | <code>pull</code> |

Takeaway

Remotes do not change the basic picture. They add another copy of the timeline that you synchronize with.